CPE 486/586: Deep Learning for Engineering Applications

06 Generative Adversarial Network

Spring 2026

Rahul Bhadani

Outline

Lecture Overview

- In this lecture we discussed generative models (briefly) and how to model distributions of data using density estimation techniques. One advantage to these generative models is that we can generate data from the distribution once we have the model.
- Our goal in this lecture is to introduce *Generative Adversarial Networks* (GANs) for generating data from a distribution $p_{data}(\mathbf{x})$. GANs are able to model complex distributions of data that would be incredible challenging to model using the density estimation approaches discussed in previous lectures.
- Modeling $p_{data}(\mathbf{x})$ is increasingly challenging for complex data, such as real scenes, people, etc.

Motivation

Discriminative vs. Generative Models

Discriminative	Generative
Focuses on the **boundary** between classes.	Focuses on the **distribution** of the data.
Maps high-dimensional input $\mathbf{x}$ to label y.	Models the process that *creates* the data.
Learns $P(y \mid \mathbf{x})$.	Learns $P(\mathbf{x}, y)$ or $P(\mathbf{x})$.
Goal: Classification and Regression.	*Strategy:* MLE or Variational Inference.

*While **Discriminative** models tell you how to tell things apart, **Generative** models give you the "recipe" for the data itself.*

GAN is a generative model.

Adversarial Machine Learning

Wikipedia Definition

Study of attacks on ML algorithms and defenses against such attacks.

- **Data Assumption:** Conventional training is based on IID assumption, but however someone can supply non-IID data.
- **Exploitation:** Adversaries may attempt to exploit a learning mechanism either to cause it to misbehave or to extract or misuse information.

While we won't delve in Secure Machine Learning, we serve this as a motivation for **Generative Adversarial Network (GAN)**.

GAN is a **two-player minimax** game where:

Generator: A counterfeiter (adversary) trying to fool you.

Discriminator: A police officer investigating and detecting counterfeit objects.

Example of a Complex $p_{data}(\mathbf{x})$

Example of a Complex $p_{data}(\mathbf{x})$

Example of a Complex $p_{data}(\mathbf{x})$



Example of a Complex $p_{data}(\mathbf{x})$

None of these people are real!

GANs

There are hundreds(?) of different types of GANs; however, this lecture only presents the vanilla GAN.

Generative Adversarial Networks

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- Access to $\{\mathbf{x}_i\}_{i=1}^{n}$
- No labels needed (not classification).

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- Access to $\{\mathbf{x}_i\}_{i=1}^{n}$
- No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- Access to $\{\mathbf{x}_i\}_{i=1}^{n}$
- No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Generator $G(\mathbf{z})$

G is the generator neural network that takes in a noise vector, $\mathbf{z}$, to produce a vector $\mathbf{x}_{fake}$. $\mathbf{x}_{fake}$ is a "fake" data sample that is generated from $\mathbf{z}$.

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- Access to $\{\mathbf{x}_i\}_{i=1}^{n}$
- No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Generator $G(\mathbf{z})$

G is the generator neural network that takes in a noise vector, $\mathbf{z}$, to produce a vector $\mathbf{x}_{fake}$. $\mathbf{x}_{fake}$ is a "fake" data sample that is generated from $\mathbf{z}$.

Discriminator $D(\mathbf{x})$

D takes vector $\mathbf{x}$ and tries to discern if $\mathbf{x}$ was sampled from $p_{data}(\mathbf{x})$ or generated by G.

Definitions

Data Distribution

$p_{data}(\mathbf{x})$: Probability distribution over the data.

- Access to $\{\mathbf{x}_i\}_{i=1}^{n}$
- No labels needed (not classification).

Latent Variable

$p_g(\mathbf{z})$: Probability distribution over a latent variable $\mathbf{z}$. Think of this distribution as being over random noise.

Generator $G(\mathbf{z})$

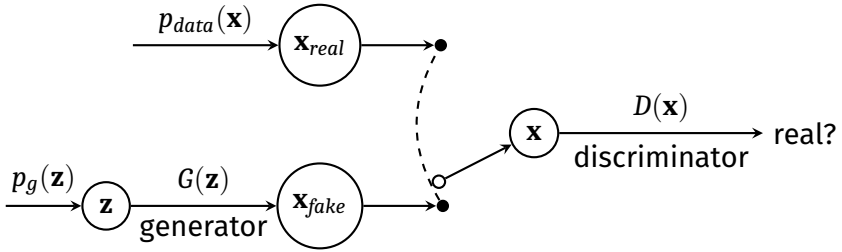
G is the generator neural network that takes in a noise vector, $\mathbf{z}$, to produce a vector $\mathbf{x}_{fake}$. $\mathbf{x}_{fake}$ is a "fake" data sample that is generated from $\mathbf{z}$.

Discriminator $D(\mathbf{x})$

D takes vector $\mathbf{x}$ and tries to discern if $\mathbf{x}$ was sampled from $p_{data}(\mathbf{x})$ or generated by G.

Let θ_g and θ_d represent the parameters for the generator and discriminator neural networks, respectively.

Generative Adversarial Net



Goodfellow, Ian J., Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. "Generative adversarial nets." Advances in neural information processing systems 27 (2014).

Generative Adversarial Net

- We have two neural network in a GAN
  - The network G's goal is to generate sample that look like they came from $p_{data}(\mathbf{x})$.
  - The network D's goal is to effectively discriminate between real samples from $p_{data}(\mathbf{x})$ and fake samples from G.
- The goals of G and D are different! We can view this scenario as a game between D and G.

We train D to maximize the probability of assigning the correct label to both training examples and samples from G. We simultaneously train G to minimize $\log(1 - D(G(z)))$.

Formally Defining the Optimization Task

The optimization of θ_g and θ_d can be cast as a $\min$-$\max$ task, which is given by

$$\min_G \max_D V(D, G) = \{\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log(D(\mathbf{x}))] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [\log(1 - D(G(\mathbf{z})))]\} \quad (1)$$

V is called as value function.

From the original paper: alternate between k steps of optimizing D and one step of optimizing G.

In the beginning G is poor, so D will reject samples with a high confidence, and $\log(1 - D(G(z)))$ saturates. So, rather than training G to minimize $\log(1 - D(G(z)))$, G is trained to maximize $\log D(G(z))$.

Formally Defining the Optimization Task

- If $\mathbf{x} \sim p_{data}(\mathbf{x})$ then the discriminator should maximize $\log D(\mathbf{x})$
- If $\mathbf{z} \sim p_g(\mathbf{z})$ then the generator seeks to generate a sample that makes the discriminator think it is a normal sample.
- We can use stochastic gradient descent and ascent to adjust the parameters of the network.

Global Optimality of $p_g = p_{\text{data}}$

If we fix the generator G, the optimal discriminator D is

$$D^*_G(\mathbf{x}) = \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})}$$

Minimax Equation with Optimal Discriminator

For a fixed G, the minimax is

$$
\begin{aligned}
C(G) &= \max_D V(D, G) \\
&= \mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log(D_G^*(\mathbf{x})) \right] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} \left[\log(1 - D_G^*(G(\mathbf{z}))) \right] \\
&= \mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log(D_G^*(\mathbf{x})) \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log(1 - D_G^*(\mathbf{x})) \right] \\
&= \mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_{\mathbf{x}}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right]
\end{aligned}
$$

Here, training objective of D can be interpreted as maximizing the log-likelihood for estimating the conditional probability $P(Y = y|\mathbf{x})$ where Y is the random variable denoting where $\mathbf{x}$ comes from p_{data} with $y = 1$ or from p_g with $y = 0$.

Formally Defining the Optimization Task

Theorem 1. The global minimum of the virtual training criterion $C(G)$ is achieved if and only if $p_g = p_{data}$. At that point, $C(G)$ achieves the value $-\log(4)$. This point is a Nash Equilibrium.

Pseudo Code for Training a GAN

for $t = 1, \ldots, T$

for $k = 1, \ldots, K$

- Sample m noise data $\mathbf{z}_1, \ldots, \mathbf{z}_m$ from $p_g(\mathbf{z})$
- Sample m data samples $\mathbf{x}_1, \ldots, \mathbf{x}_m$ from $p_{data}(\mathbf{x})$
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \left\{ \frac{1}{m} \sum_{i=1}^{m} [\log(D(\mathbf{x}_i)) + \log(1 - D(G(\mathbf{z}_i)))] \right\} \tag{2}$$

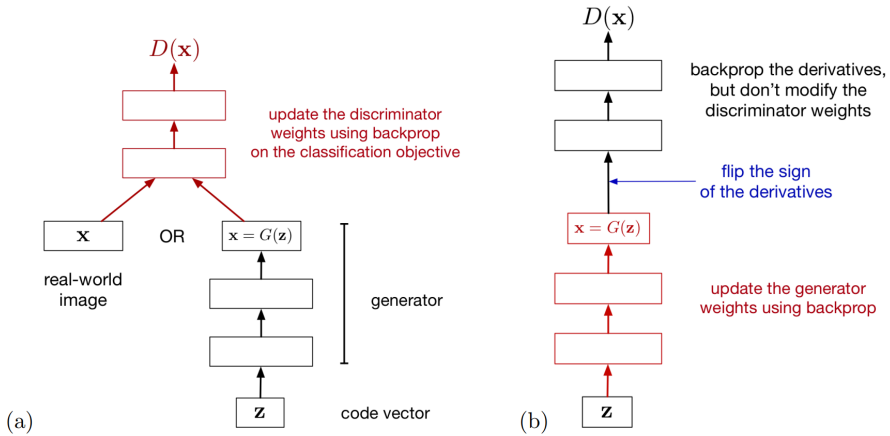
end for

- Sample m noise data $\mathbf{z}_1, \ldots, \mathbf{z}_m$ from $p_g(\mathbf{z})$
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \left\{ \frac{1}{m} \sum_{i=1}^{m} \log(1 - D(G(\mathbf{z}_i))) \right\} \tag{3}$$

end for

Formally Defining the Optimization Task



From Roger Grosse's GAN lecture notes.

On Training GANs

Challenge

One issue associated with training a GAN is the form of the loss. Observe above that the generator weights only get updated by the gradients of the second term:

$$\log(1 - D(G(\mathbf{z}))) \tag{4}$$

- The problem with this is that if the generator sample is really bad, then the discriminator's prediction is close to zero, and since $\log(1 - D)$ is very flat when $D \approx 0$ there is not enough "signal" to move the generator weights meaningfully.
- Increasing learning rates do not seem to help. This is called the *saturation problem* in GANs.
- To fix this, while updating generator weights, it is common to heuristically replace the second term in the GAN loss with $-\log D(G(\mathbf{z}))$

On Training GANs

Challenge

Stably training GANs was a challenge faced by the community for quite some time (and continues to be a challenge), and a common resolution is to use *Wasserstein GANs*.

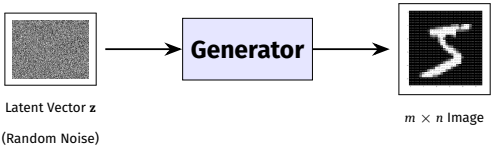
- The GAN loss is a form of distance between probability distributions (called the Jensen-Shannon divergence), and this can be generated to other distances.
- A common alternative is the Earth-mover or Wasserstein distance, leading to a different type of GAN model called Wasserstein GAN. The WGAN was shown to improve stability of the optimization process

GANs (in PyTorch)

Generators

Conceptual Flow

The Generator takes random noise (latent vector $\mathbf{z}$) and generates an $m \times n$ image.



PyTorch Implementation

```python
class Generator(nn.Module):
    def __init__(self, latent_dim=100, height=28, width=28):
        super(Generator, self).__init__()
        self.latent_dim = latent_dim
        self.height, self.width = height, width

        self.model = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 1024),
            nn.LeakyReLU(0.2),
            nn.Linear(1024, width*height),
            nn.Tanh() # Output in [-1, 1]
        )

    def forward(self, z):
        img = self.model(z)
        # Reshape to [batch, channels, H, W]
        return img.view(img.size(0), 1,
                        self.height, self.width)
```

Discriminators

Conceptual Flow

The discriminator takes generated image and guess whether it is fake or real through binary classification.

PyTorch Implementation

```python
class Discriminator(nn.Module):
    def __init__(self, height  = 28, width = 28):
        super(Discriminator, self).__init__()
        self.height, self.width = height, width

        self.model = nn.Sequential(
            nn.Linear(self.height*self.width, 1024),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),  # Regularization
            nn.Linear(1024, 512),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(256, 1),
            nn.Sigmoid()  # Output probability [0, 1]
        )

    def forward(self, img):
        img_flat = img.view(img.size(0), -1)  # Flatten
        validity = self.model(img_flat)
        return validity
```

Training Loss in GAN Implementation

Training loss is actually just Binary Cross Entropy loss effectively.

$$\mathcal{L}_{BCE}(y, \hat{y}) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Case 1: Real Sample ($y = 1$)

$$\mathcal{L}_{BCE} = -\log D(x)$$

Case 2: Fake Sample ($y = 0$)

$$\mathcal{L}_{BCE} = -\log(1 - D(G(z)))$$

Exactly the negative of the value function mentioned in the original paper.

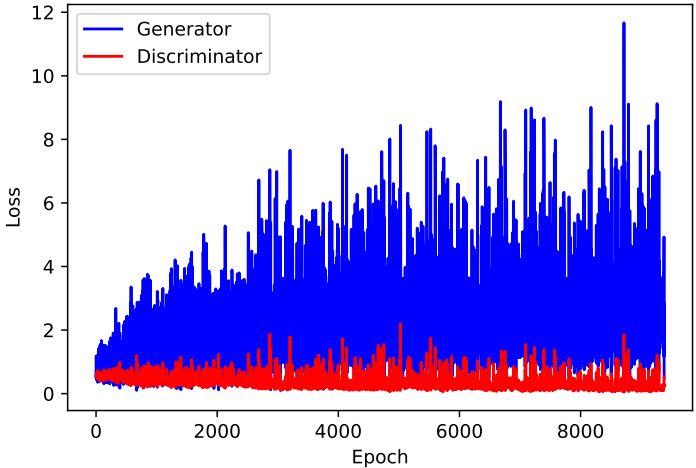
We also noted earlier that GoodFellow recommended to maximize $\log D(G(z))$ instead of minimizing $\log(1 - D(G(z)))$, becase early training: D easily detects fakes $\Rightarrow D(G(z)) \approx 0$

$$\frac{\partial}{\partial \theta} \log(1 - D(G(z))) \approx 0$$ **(Vanishing gradients!)**

Theory (Maximize)	Code (Minimize)
$\mathbb{E}[\log D(x)]$	`criterion(D(x), 1)`
$\mathbb{E}[\log(1 - D(G(z)))]$	`criterion(D(G(z)), 0)`

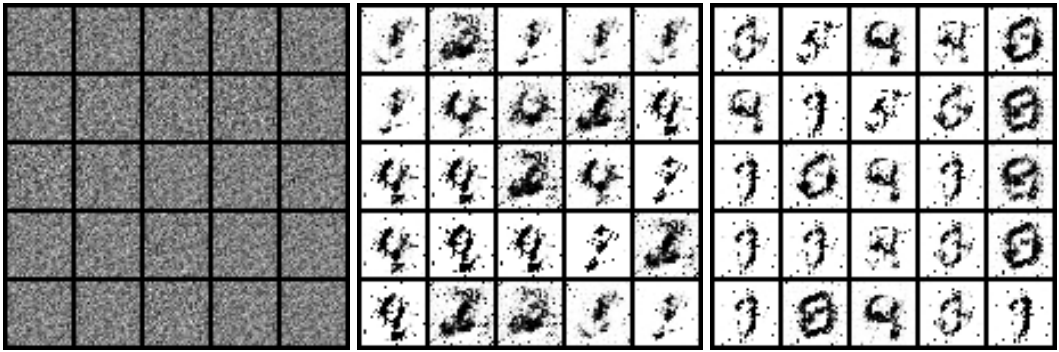
Implementation: Just flip the label!

GAN Loss


Generator
Discriminator
Loss
Epoch
0
2
4
6
8
10
12
0
2000
4000
6000
8000

Examples

GAN example



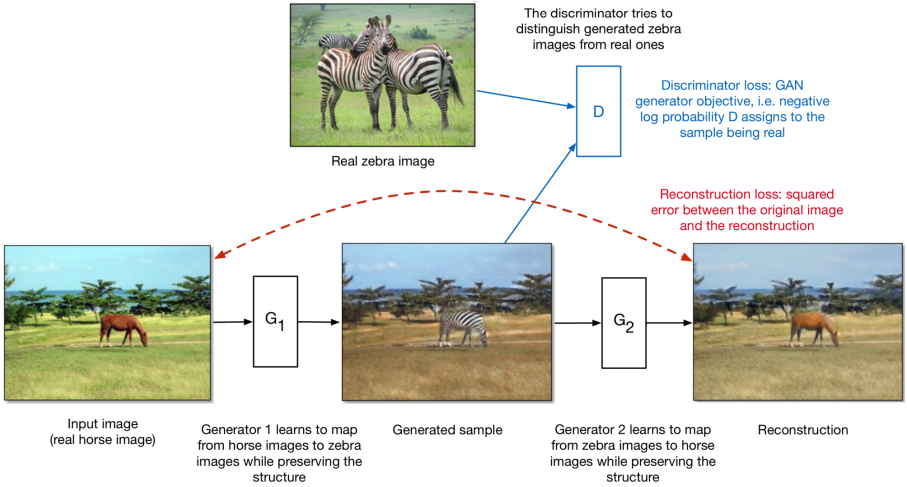
GAN example: Sketch to Figure

GAN example: Cycle GANs

- Image-to-image translation involves generating a new synthetic version of a given image with a specific modification, such as translating a summer landscape to winter

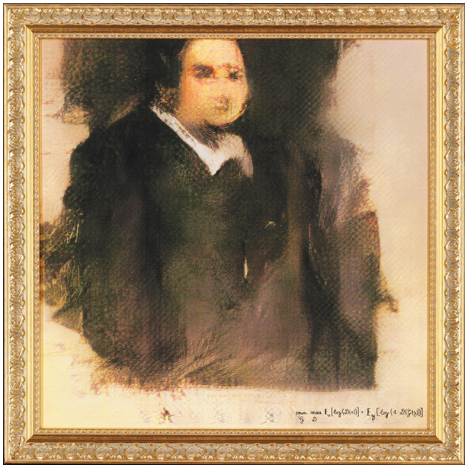
GAN example: Cycle GANs

GAN example: Cycle GANs



Total loss = discriminator loss + reconstruction loss

First AI-Generated Portrait Ever Sold at Auction Shatters Expectations: $432,500



Cosmopolitan Meets DALL-E 2

DALL-E 2: Text-to-Image Synthesis

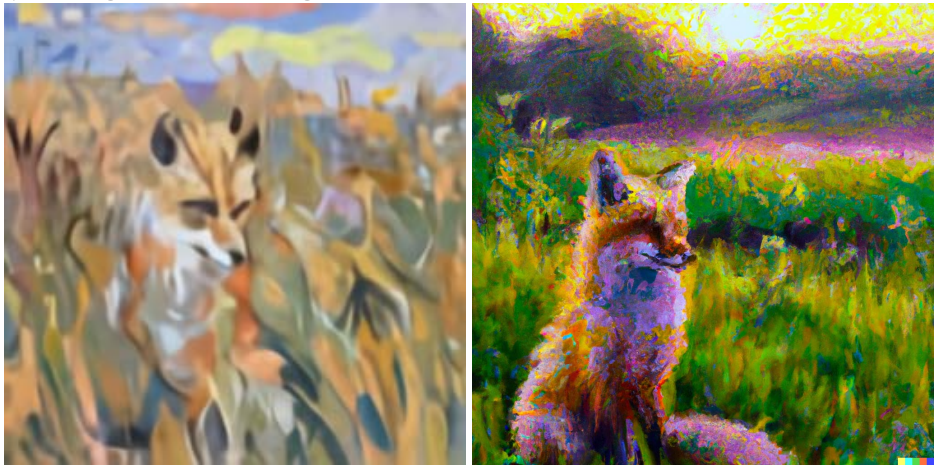
- DALL-E 2 is a new AI system that can create realistic images and art from a description in natural language. DALL-E 2 can create original, realistic images and art from a text description. It can combine concepts, attributes, and styles.
- DALL-E's uses multimodal version of GPT-3 with 12 billion parameters. The concept is that it swaps text for pixels and it is trained on trained on text-image pairs from the Internet. DALL-E 2 uses 3.5 billion parameters.

DALL-E 2

“a painting of a fox sitting in a field at sunrise in the style of Claude Monet”

DALL-E 2

"a painting of a fox sitting in a field at sunrise in the style of Claude Monet"



Challenges with GANs

Issues with GANs (Google's Comments)

- Research has suggested that if your discriminator is too good, then generator training can fail due to vanishing gradients. In effect, an optimal discriminator doesn't provide enough information for the generator to make progress.
  - The Wasserstein loss is designed to prevent vanishing gradients even when you train the discriminator to optimality.
- The generator may learn to produce a single output. The generator is always trying to find the one output that seems most plausible to the discriminator. This form of GAN failure is called *mode collapse*.
- GANs frequently fail to converge, as discussed in the module on training.

Advance Architecture in GAN

Deep Convolution GAN

Fractionally-Strided Convolutional Layer (or Transposed Convolution) is opposite of convolution operation.

In a convolution operation (for example, stride = 2), we obtain a downsampled (smaller) output of the larger input.

In fractionally-streded convolution, an upsampled (larger) output is obtained from a smaller input.

Consider $X = \begin{bmatrix} x_{00} & x_{01} \\ x_{10} & x_{11} \end{bmatrix}$ and filter as $K = \begin{bmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & k_{11} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{bmatrix}$

with stride of 2, i.e. fractional stride of 1/2, we insert, zeros to between every other input pixel. Hence, $X = \begin{bmatrix} x_{00} & 0 & x_{01} & 0 \\ 0 & 0 & 0 & 0 \\ x_{10} & 0 x_{11} & 0 & \\ 0 & 0 & 0 & 0 \end{bmatrix}$. Then we apply regular convolution operation.

Radford, Alec, Luke Metz, and Soumith Chintala. "Unsupervised representation learning with deep convolutional generative adversarial networks." arXiv preprint arXiv:1511.06434 (2015).

Deep Convolution GAN

Fractionally-Strided Convolutional Layer (or Transposed Convolution) is the opposite of the convolution operation.

Key Concepts:

- **Standard Convolution:** In a convolution operation (for example, stride = 2), we obtain a downsampled (smaller) output of the larger input.
- **Fractional Stride:** In fractionally-strided convolution, an upsampled (larger) output is obtained from a smaller input.

Radford, Alec, Luke Metz, and Soumith Chintala. "Unsupervised representation learning with deep convolutional generative adversarial networks." arXiv preprint arXiv:1511.06434 (2015).

Mathematical Example: Consider $X = \begin{bmatrix} x_{00} & x_{01} \\ x_{10} & x_{11} \end{bmatrix}$ and filter as $K = \begin{bmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & k_{11} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{bmatrix}$

We insert zeros between every other input pixel. Hence:

$$X = \begin{bmatrix} x_{00} & 0 & x_{01} & 0 \\ 0 & 0 & 0 & 0 \\ x_{10} & 0 & x_{11} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Then we apply regular convolution operation.

Deep Convolution GAN

From the original paper:

1. Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
2. Use batchnorm in both the generator and the discriminator.
3. Remove fully connected hidden layers for deeper architectures.
4. Use ReLU activation in generator for all layers except for the output, which uses Tanh.
5. Use LeakyReLU activation in the discriminator for all layers.

The DCGAN made possible vector arithmetic for image maniuplation: vector(”King”) - vector(”Man”) + vector(”Woman”) = vector(”Queen”).

Cycle GAN Architecture

It is used for image to image translation to transfer some interesting features from image to another image, as we saw earlier. Examples: Horse to Zebra, Summer to Winter, etc.
Cycle GAN introduces **Cycle Consistency Loss** in addition to adversarial Loss seen in the original GAN.
Cycle GAN has four networks:

1. $G_{A \to B}$: a generator for translating images from domain $A \to B$.
2. D_B: a discriminator if an image is real or fake in B.
3. $G_{B \to A}$: a generator for translating images from domain $B \to A$.
4. D_A: a discriminator if an image is real or fake in A.

Code

https://github.com/rahulbhadani/CPE487587_SP26/blob/master/Code/Chapter_06_Generative_Adversarial_Network.ipynb

References

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, Yoshua Bengio (2013)

Generative Adversarial Networks

Advances in Neural Information Processing Systems.

The End